

Лабораторная работа № 10

Балансировка нагрузки и доступность Web-сервисов с HAProxy

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - четыре основные виртуальные машины:
 - client01;
 - lb01;
 - web01;
 - web02;
 - дополнительная VM zabbix01 для расширенного задания;
 - Nginx;
 - HAProxy;
 - Git — для сохранения проверенной конфигурации.
-

1. Цель работы

Научиться разворачивать и проверять отказоустойчивый Web-сервис на основе HAProxy и двух Backend-серверов.

В ходе работы необходимо:

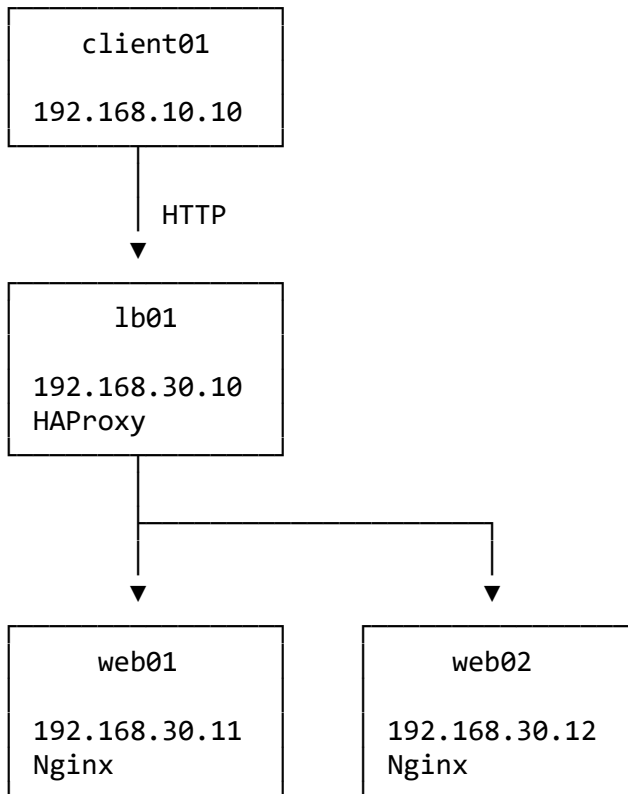
- подготовить два Nginx Backend-сервера;
- разместить на Backend разные тестовые страницы;
- создать отдельный URL /health;
- установить HAProxy;
- настроить секции global, defaults, frontend и backend;
- настроить алгоритм Round Robin;
- настроить HTTP Health Checks;
- передавать проверенный адрес клиента через X-Forwarded-For;
- проверить распределение запросов;
- смоделировать отказ одного Backend;
- смоделировать прикладной отказ с HTTP 500;
- проверить полный отказ пула и HTTP 503;

- сохранить проверенную конфигурацию в Git.
-

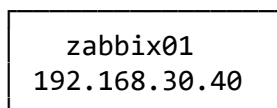
2. Схема стенда

Клиентская сеть
192.168.10.0/24

Серверная сеть
192.168.30.0/24



Дополнительно:



Маршрутизация между сетями 192.168.10.0/24 и 192.168.30.0/24 подготавливается преподавателем заранее.

В упрощённом варианте все VM можно подключить к одной внутренней сети VirtualBox, сохранив адреса lb01, web01 и web02.

3. Таблица адресации

Устройство	IP-адрес	Назначение
client01	192.168.10.10	Клиентская проверка
lb01	192.168.30.10	HAProxy
web01	192.168.30.11	Первый Nginx Backend
web02	192.168.30.12	Второй Nginx Backend
zabbix01	192.168.30.40	Мониторинг

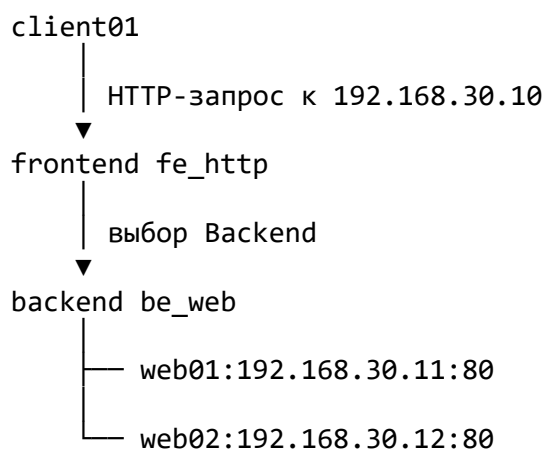
Backend-серверы должны возвращать разные страницы:

Response from web01

Response from web02

Это позволяет визуально проверить распределение запросов.

4. Логика обработки запроса



HAProxy выполняет две роли:

- Reverse Proxy — принимает запрос вместо внутренних серверов;
 - Load Balancer — выбирает один из нескольких Backend.
-

5. Базовые задания

Задание 1. Проверить исходную связность

На lb01 проверить адреса:

```
hostnamectl  
ip -br link  
ip -br addr  
ip route
```

Проверить доступность Backend:

```
ping -c 4 192.168.30.11  
ping -c 4 192.168.30.12
```

На client01 проверить доступность балансировщика:

```
ping -c 4 192.168.30.10
```

Проверить локальное решение маршрутизации:

```
ip route get 192.168.30.10
```

На всех серверах проверить время:

```
date  
timedatectl
```

Ожидаемый результат

- lb01 достигает web01 и web02;
 - client01 достигает lb01;
 - время на серверах синхронизировано;
 - отсутствуют дублирующиеся IP-адреса.
-

Задание 2. Подготовить web01

На web01 установить Nginx:

```
sudo apt update  
sudo apt install -y nginx curl
```

Создать каталог сайта:

```
sudo mkdir -p /srv/lab-web
```

Создать страницу:

```
sudo tee /srv/lab-web/index.html >/dev/null <<'EOF'  
<!doctype html>  
<html lang="ru">  
<head>  
  <meta charset="utf-8">  
  <title>web01</title>  
</head>  
<body>  
  <h1>Response from web01</h1>  
  <p>Backend address: 192.168.30.11</p>
```

```
</body>
</html>
EOF
```

Создать формат журнала с X-Forwarded-For:

```
sudo tee /etc/nginx/conf.d/lab-xff.conf >/dev/null <<'EOF'
log_format lab_xff
    '$remote_addr xff="$http_x_forwarded_for" '
    '"$request" status=$status';
EOF
```

Отключить стандартный сайт:

```
sudo rm -f /etc/nginx/sites-enabled/default
```

Создать конфигурацию Backend:

```
sudo tee /etc/nginx/sites-available/lab-backend >/dev/null <<'EOF'
server {
    listen 80 default_server;
    server_name _;

    root /srv/lab-web;
    index index.html;

    access_log /var/log/nginx/lab-access.log lab_xff;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "OK web01\n";
    }

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF
```

Включить сайт:

```
sudo ln -s \
    /etc/nginx/sites-available/lab-backend \
    /etc/nginx/sites-enabled/lab-backend
```

Проверить конфигурацию:

```
sudo nginx -t
```

Ожидается:

```
syntax is ok
test is successful
```

Применить:

```
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

Проверить:

```
curl http://127.0.0.1/
curl -i http://127.0.0.1/health
```

Ожидается:

Response from web01

и:

```
HTTP/1.1 200 OK
OK web01
```

Задание 3. Подготовить web02

На web02 установить Nginx:

```
sudo apt update
sudo apt install -y nginx curl
```

Создать каталог:

```
sudo mkdir -p /srv/lab-web
```

Создать отличающуюся страницу:

```
sudo tee /srv/lab-web/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>web02</title>
</head>
<body>
    <h1>Response from web02</h1>
    <p>Backend address: 192.168.30.12</p>
</body>
</html>
EOF
```

Создать формат журнала:

```
sudo tee /etc/nginx/conf.d/lab-xff.conf >/dev/null <<'EOF'
log_format lab_xff
    '$remote_addr xff="$http_x_forwarded_for" '
    '"$request" status=$status';
EOF
```

Отключить стандартный сайт:

```
sudo rm -f /etc/nginx/sites-enabled/default
```

Создать конфигурацию:

```
sudo tee /etc/nginx/sites-available/lab-backend >/dev/null <<'EOF'
server {
    listen 80 default_server;
    server_name _;

    root /srv/lab-web;
    index index.html;

    access_log /var/log/nginx/lab-access.log lab_xff;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "OK web02\n";
    }

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF
```

Включить сайт:

```
sudo ln -s \
/etc/nginx/sites-available/lab-backend \
/etc/nginx/sites-enabled/lab-backend
```

Проверить и применить:

```
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

Проверить:

```
curl http://127.0.0.1/
curl -i http://127.0.0.1/health
```

Ожидается:

Response from web02

и:

```
HTTP/1.1 200 OK
OK web02
```

Задание 4. Проверить Backend напрямую

На lb01 выполнить:

```
curl http://192.168.30.11/
```

```
curl http://192.168.30.12/
```

Проверить Health Endpoint:

```
curl -i http://192.168.30.11/health
```

```
curl -i http://192.168.30.12/health
```

Проверить TCP-порты:

```
nc -vz 192.168.30.11 80
```

```
nc -vz 192.168.30.12 80
```

При отсутствии nc установить:

```
sudo apt install -y netcat-openbsd
```

Ожидаемый результат

Backend	/	/health
web01	Response from web01	HTTP 200
web02	Response from web02	HTTP 200

Не следует переходить к настройке HAProxy, пока оба Backend не работают напрямую.

Задание 5. Установить и настроить HAProxy

На lb01 установить пакеты:

```
sudo apt update
```

```
sudo apt install -y haproxy socat curl
```

Проверить версию:

```
haproxy -vv | head
```

Проверить службу:

```
systemctl status haproxy --no-pager
```

Создать резервную копию исходной конфигурации:

```
sudo cp -a \  
  /etc/haproxy/haproxy.cfg \  
  /etc/haproxy/haproxy.cfg.original
```


Открыть файл:

```
sudo nano /etc/haproxy/haproxy.cfg
```

Заменить содержимое:

```
global
    log /dev/log local0
    log /dev/log local1 notice

    user haproxy
    group haproxy
    daemon

    stats socket /run/haproxy/admin.sock \
        mode 660 \
        level operator \
        expose-fd listeners

defaults
    log global
    mode http

    option httplog
    option dontlognull

    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend fe_http
    bind 192.168.30.10:80

    http-request del-header X-Forwarded-For
    http-request add-header X-Forwarded-For %[src]

    default_backend be_web

backend be_web
    balance roundrobin

    option httpchk GET /health
    http-check expect status 200

    server web01 192.168.30.11:80 \
        check inter 2s fall 3 rise 2

    server web02 192.168.30.12:80 \
        check inter 2s fall 3 rise 2
```

Назначение секций

Секция	Назначение
global	Общие параметры процесса HAProxy
defaults	Параметры по умолчанию
frontend	Приём клиентских соединений
backend	Пул серверов назначения
server	Конкретный Backend-сервер

Задание 6. Проверить конфигурацию и запустить HAProxy

Проверить синтаксис до применения:

```
sudo haproxy \
  -c \
  -f /etc/haproxy/haproxy.cfg
```

Ожидается:

```
Configuration file is valid
```

Перезапустить службу:

```
sudo systemctl restart haproxy
```

Включить автозапуск:

```
sudo systemctl enable haproxy
```

Проверить состояние:

```
systemctl status haproxy --no-pager
```

Проверить прослушивание:

```
sudo ss -lntp | grep ':80'
```

Ожидаемый адрес:

```
192.168.30.10:80
```

Порт не должен прослушиваться как:

```
0.0.0.0:80
```

Проверить журнал:

```
sudo journalctl \
  -u haproxy \
  -b \
  --since '-10 min' \
  --no-pager
```

Проверить HTTP:

```
curl -i http://192.168.30.10/
```

Задание 7. Проверить Round Robin

На client01 выполнить серию запросов.

Использовать заголовок:

Connection: close

чтобы каждый запрос создавал отдельное TCP-соединение:

```
for i in $(seq 1 10); do
    curl \
        -s \
        -H 'Connection: close' \
        http://192.168.30.10/ |
        grep -o 'Response from web0[12]'
```

done

Пример результата:

```
Response from web01
Response from web02
Response from web01
Response from web02
Response from web01
Response from web02
```

Подсчитать ответы:

```
for i in $(seq 1 20); do
    curl \
        -s \
        -H 'Connection: close' \
        http://192.168.30.10/
done |
grep -o 'Response from web0[12]' |
sort |
uniq -c
```

Пример:

```
10 Response from web01
10 Response from web02
```

Точное распределение может немного отличаться, но оба Backend должны участвовать в обработке запросов.

Почему закрывается соединение

HTTP Keep-Alive может передать несколько HTTP-запросов через одно TCP-соединение.

В таком случае серия запросов может попасть на один Backend и создать ложное впечатление, что Round Robin не работает.

Задание 8. Проверить X-Forwarded-For

На client01 отправить обычный запрос:

```
curl \
  -H 'Connection: close' \
  http://192.168.30.10/
```

На web01 и web02 посмотреть журналы:

```
sudo tail -n 10 /var/log/nginx/lab-access.log
```

Пример строки:

```
192.168.30.10 xff="192.168.10.10" "GET / HTTP/1.1" status=200
```

Здесь:

- 192.168.30.10 — адрес HAProxy, установившего соединение с Backend;
- 192.168.10.10 — проверенный исходный адрес клиента.

Проверить защиту от поддельного заголовка:

```
curl \
  -H 'X-Forwarded-For: 127.0.0.1' \
  -H 'Connection: close' \
  http://192.168.30.10/
```

Снова посмотреть журнал Backend:

```
sudo tail -n 5 /var/log/nginx/lab-access.log
```

В X-Forwarded-For не должен сохраниться поддельный адрес:

```
127.0.0.1
```

HAProxy сначала удаляет недоверенный заголовок клиента, а затем добавляет фактический адрес соединения.

Задание 9. Смоделировать отказ одного Backend

Сначала проверить состояния через Runtime Socket.

На lb01:

```
echo 'show stat' |  
sudo socat \  
    stdio \  
    /run/haproxy/admin.sock |  
grep -E 'be_web,(web01|web02)'
```

Остановить Nginx на web01:

```
sudo systemctl stop nginx
```

Проверить:

```
systemctl is-active nginx  
sudo ss -lntp | grep ':80'
```

Подождать не менее шести секунд:

```
sleep 7
```

Параметры:

```
inter 2s  
fall 3
```

означают, что HAProxy должен получить три неуспешные проверки с интервалом две секунды.

Проверить состояние на lb01:

```
echo 'show stat' |  
sudo socat \  
    stdio \  
    /run/haproxy/admin.sock |  
grep -E 'be_web,(web01|web02)'
```

В строке web01 должен появиться статус:

DOWN

Проверить сервис с client01:

```
for i in $(seq 1 6); do  
    curl \  
        -s \  
        -H 'Connection: close' \  
        http://192.168.30.10/ |  
        grep -o 'Response from web0[12]'  
done
```

Все запросы должны возвращать:

Response from web02

Вывод

Сервис продолжает работать, но резервирование потеряно.

Это состояние деградации, а не полный отказ.

Восстановить web01

На web01:

```
sudo systemctl start nginx
```

Проверить:

```
curl -i http://127.0.0.1/health
```

Подождать:

```
sleep 5
```

Параметр:

```
rise 2
```

означает, что Backend возвращается в пул после двух успешных проверок.

На lb01:

```
echo 'show stat' |  
  sudo socat \  
    stdio \  
    /run/haproxy/admin.sock |  
  grep -E 'be_web,(web01|web02)'
```

Повторить серию запросов и убедиться, что снова отвечают оба сервера.

Задание 10. Смоделировать прикладной отказ

TCP-порт может оставаться открытым, хотя приложение возвращает ошибку.

На web01 создать резервную копию конфигурации:

```
sudo cp -a \  
  /etc/nginx/sites-available/lab-backend \  
  /etc/nginx/sites-available/lab-backend.before-health-failure
```

Открыть:

```
sudo nano /etc/nginx/sites-available/lab-backend
```

Изменить блок:

```
location = /health {
    access_log off;
    default_type text/plain;
    return 200 "OK web01\n";
}
```

на:

```
location = /health {
    access_log off;
    default_type text/plain;
    return 500 "FAILED web01\n";
}
```

Проверить:

```
sudo nginx -t
```

Применить:

```
sudo systemctl reload nginx
```

Проверить главную страницу:

```
curl -i http://127.0.0.1/
```

Она продолжает возвращать:

```
HTTP/1.1 200 OK
```

Проверить Health Endpoint:

```
curl -i http://127.0.0.1/health
```

Ожидается:

```
HTTP/1.1 500 Internal Server Error
```

Подождать:

```
sleep 7
```

На lb01 проверить состояние:

```
echo 'show stat' |
sudo socat \
    stdio \
    /run/haproxy/admin.sock |
grep -E 'be_web,(web01|web02)'
```

web01 должен перейти в состояние DOWN, хотя TCP 80 продолжает прослушиваться.

Проверить:

```
for i in $(seq 1 6); do
    curl \
```

```
-s \  
-H 'Connection: close' \  
http://192.168.30.10/ |  
    grep -o 'Response from web0[12]'  
done
```

Все запросы должны обрабатываться web02.

Вывод

Обычная TCP-проверка могла бы считать web01 доступным.

HTTP Health Check обнаруживает прикладной отказ по коду HTTP 500.

Восстановить Health Endpoint

На web01:

```
sudo cp -a \  
    /etc/nginx/sites-available/lab-backend.before-health-failure \  
    /etc/nginx/sites-available/lab-backend
```

Проверить и применить:

```
sudo nginx -t  
sudo systemctl reload nginx
```

Проверить:

```
curl -i http://127.0.0.1/health
```

После двух успешных Health Checks web01 должен вернуться в пул.

Задание 11. Проверить полный отказ пула

Остановить Nginx на обоих Backend:

На web01:

```
sudo systemctl stop nginx
```

На web02:

```
sudo systemctl stop nginx
```

Подождать:

```
sleep 7
```

На client01 выполнить:

```
curl -i http://192.168.30.10/
```


Ожидаемый результат:

HTTP/1.1 503 Service Unavailable

На lb01 проверить:

```
echo 'show stat' |  
sudo socat \  
    stdio \  
    /run/haproxy/admin.sock |  
grep -E 'be_web,(web01|web02)'
```

Оба Backend должны быть DOWN.

Вывод

Состояние	Результат
Один Backend DOWN	Сервис продолжает работать
Оба Backend DOWN	Клиент получает HTTP 503

Восстановить сервис

На web01:

```
sudo systemctl start nginx
```

На web02:

```
sudo systemctl start nginx
```

Подождать:

```
sleep 5
```

Проверить:

```
curl -i http://192.168.30.10/
```

Ожидается HTTP 200.

Задание 12. Сохранить конфигурацию в Git

Использовать репозиторий:

~/linux-infrastructure

созданный в лабораторной работе № 6.

Перейти в репозиторий:

```
cd ~/linux-infrastructure
```

Создать каталог:

```
mkdir -p configs/haproxy
```

Скопировать конфигурацию:

```
sudo cat /etc/haproxy/haproxy.cfg \  
> configs/haproxy/haproxy.cfg
```

Проверить синтаксис копии:

```
sudo haproxy \  
-c \  
-f configs/haproxy/haproxy.cfg
```

Проверить потенциальные секреты:

```
grep -RIInE \  
--exclude-dir=.git \  
'password|passwd|token|secret|private[_-]?key|stats auth' \  
configs/haproxy || true
```

Проверить изменения:

```
git status  
git diff
```

Добавить только нужный файл:

```
git add configs/haproxy/haproxy.cfg
```

Проверить будущий Commit:

```
git diff --staged
```

Создать Commit:

```
git commit \  
-m "Add validated HAProxy web load balancer configuration"
```

Проверить:

```
git log --oneline --decorate -5  
git status
```

Ожидается чистый Working Tree.

6. Приёмочная матрица

№	Проверка	Ожидаемый результат	Фактический результат
1	web01 напрямую	Response from web01	

№	Проверка	Ожидаемый результат	Фактический результат
2	web02 напрямую	Response from web02	
3	/health на web01	HTTP 200	
4	/health на web02	HTTP 200	
5	haproxy -c	Configuration file is valid	
6	ss -lntp	HAProxy слушает 192.168.30.10:80	
7	Серия запросов	Отвечают оба Backend	
8	Поддельный XFF	Заменён реальным адресом клиента	
9	Nginx web01 остановлен	Запросы обслуживает web02	
10	/health web01 возвращает 500	web01 исключён из пула	
11	Оба Backend остановлены	HTTP 503	
12	Backend восстановлены	HTTP 200 и Round Robin	
13	Конфигурация сохранена в Git	Commit создан	

7. Дополнительные задания

Дополнительное задание 1. Сравнить Round Robin и Least Connections

Создать резервную копию:

```
sudo cp -a \
/etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.before-leastconn
```

В секции backend be_web заменить:

```
balance roundrobin
```

на:

```
balance leastconn
```

Проверить:

```
sudo haproxy \
-c \
-f /etc/haproxy/haproxy.cfg
```

Применить:

```
sudo systemctl reload haproxy
```

Проверить состояние:

```
systemctl status haproxy --no-pager
```

Вывод

Least Connections выбирает Backend с меньшим количеством активных соединений.

Он не измеряет непосредственно:

- загрузку CPU;
- использование RAM;
- очередь приложения;
- скорость диска.

Для коротких HTTP-запросов разница с Round Robin может быть почти незаметна.

Вернуть исходную конфигурацию:

```
sudo cp -a \  
  /etc/haproxy/haproxy.cfg.before-leastconn \  
  /etc/haproxy/haproxy.cfg
```

Проверить и применить:

```
sudo haproxy \  
  -c \  
  -f /etc/haproxy/haproxy.cfg
```

```
sudo systemctl reload haproxy
```

Дополнительное задание 2. Настроить TLS Termination

На lb01 создать каталог:

```
sudo install -d -m 700 /etc/haproxy/certs
```

Создать учебный сертификат:

```
sudo openssl req \  
  -x509 \  
  -newkey rsa:2048 \  
  -nodes \  
  -days 30 \  
  -keyout /tmp/lb01.key \  
  -out /tmp/lb01.crt \  
  -subj '/CN=site.example.test'
```

Объединить сертификат и Private Key:

```
sudo sh -c \  
  'cat /tmp/lb01.crt /tmp/lb01.key > /etc/haproxy/certs/site.pem'
```

Ограничить права:

```
sudo chmod 600 /etc/haproxy/certs/site.pem
sudo chown root:root /etc/haproxy/certs/site.pem
```

Удалить временные файлы:

```
sudo rm -f /tmp/lb01.crt /tmp/lb01.key
```

Добавить Frontend:

```
frontend fe_https
    bind 192.168.30.10:443 \
        ssl \
        crt /etc/haproxy/certs/site.pem

    mode http

    http-request del-header X-Forwarded-For
    http-request add-header X-Forwarded-For %[src]
    http-request set-header X-Forwarded-Proto https

    default_backend be_web
```

Проверить:

```
sudo haproxy \
    -c \
    -f /etc/haproxy/haproxy.cfg
```

Применить:

```
sudo systemctl reload haproxy
```

Проверить порт:

```
sudo ss -lntp | grep ':443'
```

Проверить TLS:

```
openssl s_client \
    -connect 192.168.30.10:443 \
    -servername site.example.test
```

Проверить HTTPS:

```
curl -k \
    https://192.168.30.10/
```

Схема

Client HTTPS

↓

HAProxy TLS Termination



Backend HTTP

Передача HTTP между HAProxy и Backend допустима только в доверенной внутренней сети и в соответствии с принятой политикой.

Дополнительное задание 3. Настроить мониторинг HAProxy в Zabbix

На lb01 установить Agent 2 и настроить его по аналогии с лабораторной работой № 8.

Создать скрипт подсчёта доступных Backend:

```
sudo nano /usr/local/sbin/haproxy-backends-up.sh
```

Добавить:

```
#!/usr/bin/env bash

set -euo pipefail

readonly SOCKET='/run/haproxy/admin.sock'

if [[ ! -S $SOCKET ]]; then
    printf '0\n'
    exit 0
fi

echo 'show stat' |
socat stdio "$SOCKET" |
awk -F, '
    $1 == "be_web" &&
    ($2 == "web01" || $2 == "web02") &&
    $18 == "UP" {
        count++
    }
    END {
        print count + 0
    }
    ,
```

Назначить права:

```
sudo chown root:root \
    /usr/local/sbin/haproxy-backends-up.sh
```

```
sudo chmod 755 \
    /usr/local/sbin/haproxy-backends-up.sh
```

Проверить:

```
sudo bash -n \  
/usr/local/sbin/haproxy-backends-up.sh
```

Для доступа пользователя zabbix к Runtime Socket может потребоваться:

- добавить zabbix в группу haproxy;
- либо использовать отдельный Root-owned Wrapper;
- либо создать безопасный State File через systemd timer.

Не выдавать пользователю zabbix неограниченный sudo.

Создать UserParameter:

```
UserParameter=custom.haproxy.backends_up,/usr/local/sbin/haproxy-backends-up.sh
```

Создать Item:

Поле	Значение
Name	HAProxy backends UP
Key	custom.haproxy.backends_up
Type	Zabbix agent
Type of information	Numeric unsigned
Update interval	30s

Создать Triggers:

Потеря резервирования

```
last(/lb01/custom.haproxy.backends_up)=1
```

Severity:

Average

Полный отказ пула

```
last(/lb01/custom.haproxy.backends_up)=0
```

Severity:

Disaster

Дополнительно настроить:

- состояние службы HAProxy;
- локальный TCP 80;
- локальный TCP 443;
- пользовательский HTTP Web Scenario через lb01.

Вывод

Мониторинг должен различать:

- один Backend DOWN — сервис работает без резервирования;
 - оба Backend DOWN — сервис полностью недоступен.
-

8. Основные команды

Nginx Backend

```
systemctl status nginx
sudo nginx -t
sudo ss -lntp | grep ':80'
curl -i http://127.0.0.1/
curl -i http://127.0.0.1/health
```

HAProxy

```
sudo haproxy \
  -c \
  -f /etc/haproxy/haproxy.cfg
```

```
systemctl status haproxy
sudo systemctl reload haproxy
sudo ss -lntp | grep ':80'
sudo journalctl -u haproxy -b
```

Runtime Socket

```
echo 'show stat' |
  sudo socat \
    stdio \
    /run/haproxy/admin.sock
```

HTTP-проверка

```
curl -i http://192.168.30.10/

for i in $(seq 1 10); do
  curl \
    -s \
    -H 'Connection: close' \
    http://192.168.30.10/
done
```

Проверка TLS

```
openssl s_client \
  -connect 192.168.30.10:443 \
  -servername site.example.test
```

9. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему и таблицу адресации.
3. Различие Reverse Proxy и Load Balancer.
4. Конфигурацию frontend `fe_http`.
5. Конфигурацию backend `be_web`.
6. Результат `haproxy -c`.
7. Результат `ss -lntp`.
8. Результат серии Round Robin-запросов.
9. Результат проверки X-Forwarded-For.
10. Состояние при отказе `web01`.
11. Состояние при HTTP 500 на `/health`.
12. Результат полного отказа и HTTP 503.
13. Заполненную приёмочную матрицу.
14. Commit с проверенной конфигурацией.
15. Краткий вывод по работе.

В отчёт не включаются:

- Private Key TLS;
 - полное содержимое PEM-файла;
 - реальные пароли Stats Page;
 - закрытые данные инфраструктуры;
 - секреты из Git-репозитория.
-

10. Чек-лист выполнения

- ☐ `web01` доступен напрямую.
- ☐ `web02` доступен напрямую.
- ☐ Backend возвращают разные страницы.
- ☐ `/health` возвращает HTTP 200.
- ☐ HAProxy установлен на `lb01`.
- ☐ Используется `mode http`.
- ☐ Frontend привязан к конкретному IP.
- ☐ Настроены таймауты.
- ☐ Используется Round Robin.
- ☐ Настроен HTTP Health Check.
- ☐ Настроены `inter`, `fall` и `rise`.

- ☐ Клиентский X-Forwarded-For удаляется.
 - ☐ HAProxy добавляет проверенный адрес клиента.
 - ☐ Конфигурация проходит haproxy -c.
 - ☐ HAProxy слушает 192.168.30.10:80.
 - ☐ Запросы распределяются между Backend.
 - ☐ При отказе одного Backend сервис работает.
 - ☐ HTTP 500 на /health исключает Backend из пула.
 - ☐ При отказе обоих Backend возвращается HTTP 503.
 - ☐ После восстановления оба Backend возвращаются в пул.
 - ☐ Проверенная конфигурация сохранена в Git.
-

11. Контрольные вопросы

1. Чем Reverse Proxy отличается от Load Balancer?
2. Почему для HTTP Health Check используется mode http?
3. Чем TCP Health Check отличается от HTTP Health Check?
4. Почему серия запросов Round Robin выполняется с Connection: close?
5. Для чего используются параметры inter, fall и rise?
6. Почему нельзя доверять входящему X-Forwarded-For от клиента?
7. Как отличается деградация сервиса от полного отказа?
8. Почему два Backend не устраняют SPOF самого HAProxy?